

Apuntes Día 2

CURSO DE DESARROLLO CON IA

El Nuevo Programador

Índice

El Nuevo Programador	3
03 - Harness engineering avanzado	3
Comandos personalizados	3
Agent Skills	4
MCP (Model Context Protocol)	7
04 - Spec-Driven Development (SDD)	9
¿Qué es?	9
Taxonomía SDD	10
EARS (Easy Approach to Requirements Syntax)	11
Estructura SDD	11
SDD paso a paso	11
Paso 1: Constitución	12
Paso 2: Especificación (spec.md)	13
Paso 3: Clarificación	14
Paso 4: Planificación (plan.md)	14
Paso 5: Tareas (tasks.md)	14
Paso 6: Implementación	14
Paso 7: Validación	14
Automatización de SDD mediante Skills y Comandos	15
Súmate a nuestro directo del día 3	20

El Nuevo Programador

00 - Fundamentos de programación

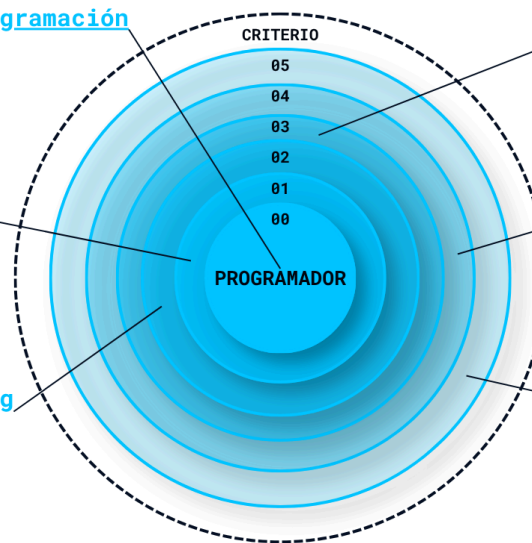
Lógica, lenguajes, frameworks, datos, arquitectura, texting, depuración...

01 - Fundamentos de IA

LLM
Modelos
Herramientas
Prompting

02 - Harness engineering

Agentes
Guardarrailes
Contexto
Plan



03 - Harness avanzado

Comandos personalizados
Agent Skills
Model Context Protocol (MCP)

04 - Spec-driven development

Especificaciones
Taxonomía SDD
Estructura
Criterios de aceptación

05 - Multiagentes y subagentes

Agentes personalizados
Subagentes personalizados
Sistemas multiagente

03 - Harness engineering avanzado

Comandos personalizados

Custom command: es un prompt reutilizable que ejecutas escribiendo `/<nombre comando>`. Permite repetir tareas habituales sin volver a escribir las instrucciones.

Creación

1. Crea un archivo Markdown en `.opencode/commands/`
2. Añade una descripción y las instrucciones
3. Reinicia **OpenCode** (`/restart` o `/exit`) y ejecuta el comando creado

<https://opencode.ai/v2/docs/commands/>

Ejemplo de comandos

`.opencode/commands/feature.md`

None

description: Planifica una nueva funcionalidad antes de tocar código
agent: plan

Quiero añadir esta funcionalidad: \$ARGUMENTS

Antes de escribir código, prepárame un plan con:

1. Cómo la vas a implementar, respetando las reglas de AGENTS.md.
2. Qué archivos vas a modificar y qué cambia en cada uno.
3. Los casos límite y las dudas que debo decidir yo antes de empezar.
4. Qué actualizarías en AGENTS.md y en MEMORY.md.

Ten en cuenta el estado actual del proyecto: @MEMORY.md

No modifiques ningún archivo hasta que apruebe el plan.

Uso del comando

None

/feature mostrar el total de minutos estudiados esta semana

Agent Skills

Skill: es un conjunto de instrucciones reutilizables que enseña al agente cómo realizar una tarea concreta. El agente puede cargar las instrucciones completas cuando las necesita (o manualmente utilizando / o @ según el agente utilizado). La diferencia principal entre un skill (habilidad) y un comando es que los comandos nunca son ejecutados por el agente por iniciativa propia.

Creación

Crea un archivo Markdown en **.opencode (o .agents)/skills/nombre_skill/**

1. Añade un archivo **SKILL.md** con instrucciones
2. Reinicia **OpenCode** y ejecuta el comando **/skills** para ver los skills instalados

<https://agentskills.io>

<https://opencode.ai/v2/docs/skills>

Ejemplo de Skill de manejo de fechas para nuestro Diario de Estudio

.agents/skills/local-dates/SKILL.md

```
None
---
name: local-dates
description: Úsala siempre que escribas, modifiques o revises código que trabaje con
fechas, días, semanas o rachas en el Diario de Estudio.
---

# Fechas locales en el Diario de Estudio

Las fechas son la mayor fuente de bugs de este proyecto. Sigue esta guía siempre que
toques código con fechas.

## Reglas
- Las fechas se guardan como texto "AAAA-MM-DD" en la zona horaria local del usuario.
- Para obtener el día de hoy, construye el texto con getFullYear(), getMonth() + 1 y
getDate(), rellenando con ceros.
- Para convertir "AAAA-MM-DD" en fecha, usa new Date(año, mes - 1, día). Nunca new
Date("AAAA-MM-DD"): se interpreta en UTC.
- Nunca uses toISOString() para obtener el día: devuelve la fecha en UTC.
- Para sumar o restar días, usa setDate(getDate() ± n), nunca milisegundos (24 h no
siempre es un día por los cambios de hora).
- Reutiliza las funciones de fechas que ya existan en app.js antes de crear otras nuevas.

## Checklist de revisión
- [ ] ¿Algún toISOString() o new Date("AAAA-MM-DD")?
- [ ] ¿Algún cálculo con 86400000 milisegundos?
- [ ] ¿Qué pasa con una sesión registrada a las 00:30?
- [ ] ¿Qué pasa el día del cambio de hora (último domingo de marzo y de octubre)?
- [ ] ¿Se ignoran las fechas futuras donde corresponde?

## Al terminar
Indica qué puntos del checklist has comprobado y cómo.
```

Uso de la skill

```
None
/feature mostrar cuántos días he estudiado este mes. ¿Vas a utilizar alguna skill?
```

Skill de terceros

Utilizando plataformas como skills.sh, podemos dotar al agente de habilidades específicas de forma modular. Esto mantiene el contexto principal limpio.

Para instalar skills en nuestro agente, debemos instalar el paquete **agent-skills** (<https://www.skills.sh/docs>)

```
None
npx skills add vercel-labs/agent-skills
```

Una vez instalado, ya podremos instalar otras skills de la misma web, por ejemplo:

frontend-design (<https://www.skills.sh/anthropics/skills/frontend-design>)

None

```
npx skills add https://github.com/anthropics/skills --skill frontend-design
```

Durante la instalación, podremos elegir los agentes para los cuales instalaremos la skill, y el alcance del mismo (a nivel global, o nivel proyecto).

Ejemplo de uso Skill frontend-design

None

```
@frontend-design con lo que sabes mejora el diseño de la web
```

Starter Pack Skills

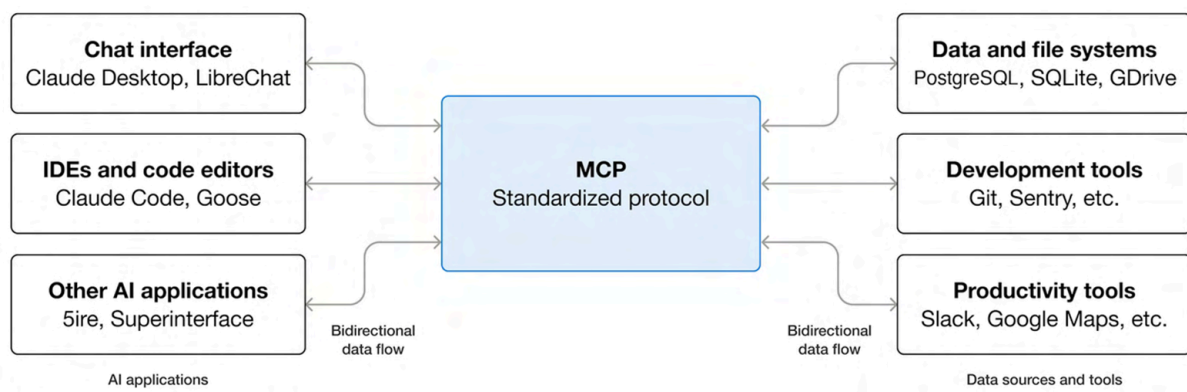
- [find-skills \(Vercel\)](#): busca e instala skills del ecosistema cuando necesitas una capacidad nueva.
- [grill-me \(Matt Pocock\)](#): te hace preguntas sobre tu plan hasta cubrir todos los casos antes de implementar.
- [frontend-design \(Anthropic\)](#): crea interfaces con un diseño cuidado y profesional.
- [web-design-guidelines \(Vercel\)](#): revisa la interfaz y detecta problemas de accesibilidad, UX y diseño.
- [systematic-debugging \(Jesse Vincent / obra\)](#): depura con método: reproducir, aislar, formular hipótesis y verificar.

MCP (Model Context Protocol)

Es un protocolo que permite conectar al agente con herramientas y datos externos. Un servidor MCP proporciona esas capacidades: consultar documentación, acceder a servicios.

Ventajas

- ✓ Estándar abierto
- ✓ Seguridad por diseño
- ✓ Reutilizable entre IDEs
- ✓ Ecosistema creciente



<https://modelcontextprotocol.io>

Creación

1. Crea o edita **opencode.json** en la raíz del proyecto.
2. Añade la configuración del servidor mcp que quieres agregar.
3. Reinicia **OpenCode** y ejecuta el comando **/mcps** para ver los MCPs instalados. Puedes activar o desactivar los MCPs.

Documentación Oficial OpenCode sobre MCP:

- <https://opencode.ai/v2/docs/mcp-servers>

Directorio de MCPs:

- <https://registry.modelcontextprotocol.io>
- <https://mcp.directory>
- <https://mcpservers.org/es>

MCPs esenciales



[Chrome DevTools \(Google\)](#)

El agente abre tu web en el navegador, la prueba, lee la consola y hace capturas.



[Context7 \(Upstash\)](#)

Le da al agente la documentación actualizada de cualquier librería o framework.



[GitHub \(GitHub\)](#)

Gestiona repositorios, issues y pull requests desde el agente.



[Figma \(Figma\)](#)

Convierte diseños de Figma en código usando el contexto real del diseño.



[Supabase \(Supabase\)](#)

Crea y consulta bases de datos directamente desde el agente.

Instalar Chrome DevTools y Context7 en nuestro Diario de Estudio opencode.json

```
JSON
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "chrome-devtools": {
      "type": "local",
      "command": [
        "npx",
        "-y",
        "chrome-devtools-mcp@latest",
        "--no-usage-statistics"
      ]
    },
    "context7": {
      "type": "remote",
      "url": "https://mcp.context7.com/mcp",
      "headers": {
        "CONTEXT7_API_KEY": "<TU APIKEY DE CONTEXT7>"
      }
    }
  }
}
```

Uso del MCP de Chrome DevTools mediante prompt

None

Usa Chrome DevTools para probar el Diario de Estudio:

1. Abre index.html en Chrome.
2. Registra tres sesiones: hoy, ayer y anteayer.
3. Comprueba que la racha muestra 3 y que la mejor racha es correcta.
4. Revisa la consola por si hay errores.
5. Haz una captura en tamaño móvil (375 px de ancho).

Dime qué has comprobado y si has encontrado algún problema.

Mejorar la capacidad de testing del agente mediante Chrome DevTools

Actualizar la información existente en **AGENTS.md** (sección “Verificación”)

Reemplazar

None

- No hay tests ni lint. Probar abriendo `index.html` en el navegador.

Por

None

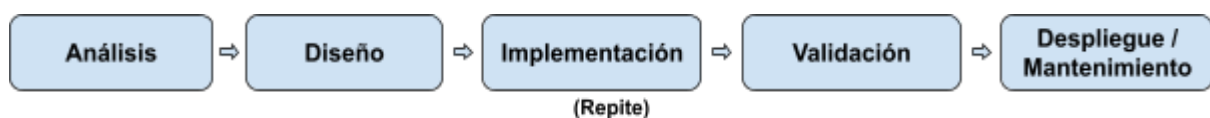
- No hay tests automáticos. Después de cada cambio, verifica con el MCP de Chrome DevTools: abre `index.html`, prueba la funcionalidad, revisa la consola y comprueba la vista móvil.

04 - Spec-Driven Development (SDD)

¿Qué es?

Es una forma de desarrollar software en la que primero defines qué debe hacer el sistema, sus requisitos, límites y criterios de aceptación, y utilizas esa especificación para guiar la implementación y comprobar el resultado.

Flujo de desarrollo clásico

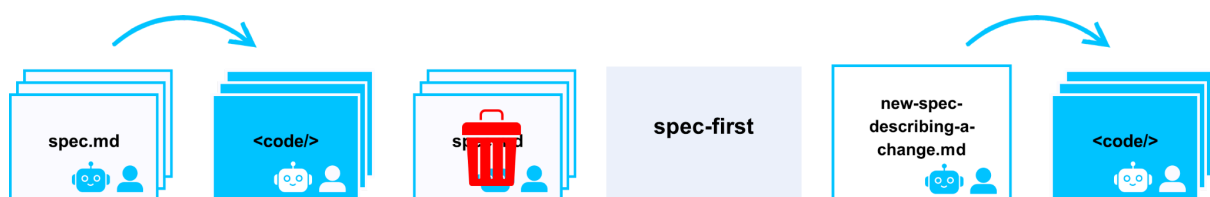


Flujo de desarrollo con SDD

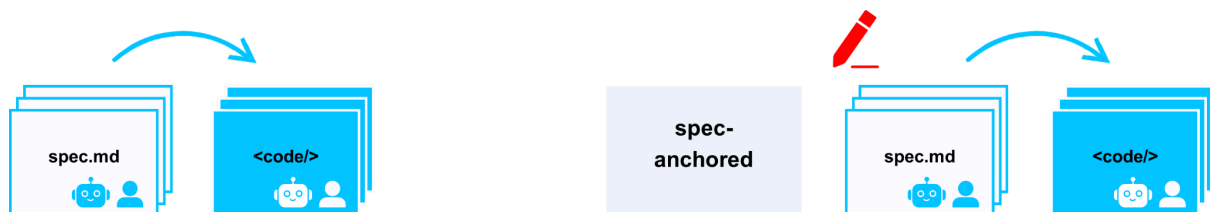


Taxonomía SDD

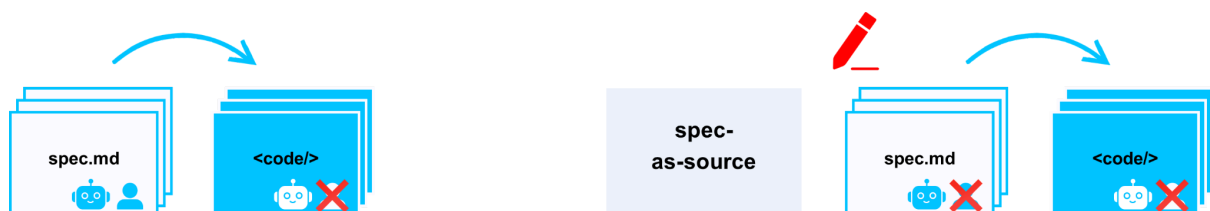
- **Spec-first:** escribes la especificación, generas el código una vez, y a partir de ahí editas el código directamente. La especificación fue el punto de partida y muere.



- **Spec-anchored:** la spec se mantiene viva: conservas y actualizas la especificación para guiar la evolución del código. Es el más recomendable para empezar.



- **Spec-as-source:** editas únicamente la especificación y el código se genera a partir de ella y no se modifica manualmente.



EARS (Easy Approach to Requirements Syntax)

EARS: es una forma de escribir requisitos con plantillas fijas para que no sean ambiguos.

Por qué funciona tan bien con IA:

- Cada requisito se puede verificar, así que se convierte casi directamente en un test.
- El SI... ENTONCES obliga a pensar en los errores, que es justo lo que la IA suele olvidar.
- Una frase = un comportamiento. Nada de párrafos donde el agente tenga que interpretar.

Estructura SDD

```
None
project/
├── .opencode/, .agents/...
├── AGENTS.md, MEMORY.md
├── docs/
│   └── constitution.md
├── specs/
│   ├── 001-nombre-spec/
│   │   ├── spec.md
│   │   ├── plan.md
│   │   └── tasks.md
│   ├── 002-nombre-spec/...
│   └── 003-nombre-spec/...
├── tests/
└── <CÓDIGO DEL PROYECTO>
```

SDD paso a paso

- **Paso 1:** Constitución (una vez por proyecto - **constitution.md**)
- **Paso 2:** Especificación (**spec.md**)
- **Paso 3:** Clarificación
- **Paso 4:** Planificación (**plan.md**)
- **Paso 5:** Tareas (**tasks.md**)
- **Paso 6:** Implementación
- **Paso 7:** Validación
- **Loop al paso 2:** Mantenimiento

Paso 1: Constitución

Ejemplo de Prompt para crear el archivo constitution.md

None

Vamos a crear la constitución del Diario de Estudio. Es un proyecto que ya existe: lee AGENTS.md, MEMORY.md y el código antes de proponer nada. Es un proyecto educativo que debe poder mantener alguien que empieza a programar.

Proponme un docs/constitution.md con 6 principios innegociables, cortos y verificables, que cubran: simplicidad del stack, relación entre spec y código, separación entre lógica e interfaz, política de tests (sin instalar dependencias), protección de los datos del usuario e idioma del código y los textos. Máximo 15 líneas. Espera mi aprobación.

docs/constitution.md de nuestro Diario de Estudio

None

```
# Constitución – Diario de Estudio
```

Principios innegociables. Toda spec, plan y tarea debe cumplirlos.

1. **Simplicidad primero**: HTML, CSS y JS puros. Sin dependencias ni build. Funciona abriendo index.html con doble clic.
2. **La spec manda**: nada se implementa si no está en la spec activa. Si falta una decisión, se para y se pregunta.
3. **Lógica separada de interfaz**: los cálculos (fechas, rachas, estadísticas) son funciones puras, sin DOM ni localStorage, que reciben "hoy" como parámetro.
4. **Tests como puerta**: la lógica se prueba con `node --test`, sin instalar paquetes. Prohibido avanzar con tests en rojo.
5. **Los datos del usuario son sagrados**: localStorage con compatibilidad hacia atrás y fechas siempre en hora local. Nunca se pierde una sesión.
6. **Idioma**: código en inglés; interfaz y documentación en español.

Dentro de nuestro **AGENTS.md** agregamos las siguientes líneas para hacer referencia a SDD:

None

```
## Comandos
```

```
- Tests: `node --test`
```

```
## Reglas
```

```
- Lee `docs/constitution.md` y la spec activa (`specs/NNN-*/`) antes de tocar código.
```

Paso 2: Especificación (spec.md)

Ejemplo de Prompt para crear una nueva especificación

None

NO escribas código en ningún momento. Vamos a redactar la especificación de una nueva funcionalidad del Diario de Estudio. Lee docs/constitution.md.

Idea inicial: un mapa de calor tipo GitHub que muestre los días estudiados de las últimas semanas, donde cuantos más minutos estudiados, más intenso es el color del día.

Tu trabajo:

1. Hazme preguntas de UNA en UNA para eliminar ambigüedades (casos límite, comportamiento con errores, qué queda fuera de esta versión). Máximo 5 preguntas.
2. Con mis respuestas, genera specs/001-heat-map/spec.md con esta estructura: contexto y objetivo, usuarios, historias de usuario, requisitos funcionales numerados (RF-x) con criterios de aceptación en notación EARS en español, requisitos no funcionales, casos límite, fuera de alcance, criterios de finalización y dudas abiertas marcadas como [NECESITA ACLARACIÓN].
3. El QUÉ y el POR QUÉ. Nada de stack, arquitectura ni nombres de archivos: eso irá en el plan.

Plantilla genérica para especificaciones (spec.md)

None

Spec NNN - <Nombre de la funcionalidad>

Contexto y objetivo

<Qué problema resuelve y por qué merece la pena. Un párrafo.>

Usuarios / actores

<Quién lo usa.>

Historias de usuario

- H1: Como <rol> quiero <acción> para <beneficio>.

Requisitos funcionales (criterios de aceptación en EARS)

- RF-1: CUANDO <evento>, EL SISTEMA <respuesta> (salida/resultado esperado).

- RF-2: SI <condición no deseada>, ENTONCES EL SISTEMA <respuesta>.

- RF-3: MIENTRAS <estado>, EL SISTEMA <respuesta>.

- RF-4: EL SISTEMA <comportamiento permanente>.

Requisitos no funcionales

<Solo los que apliquen: rendimiento, seguridad, plataformas, idioma...>

Casos límite

<Vacíos, duplicados, datos corruptos, límites, concurrencia...>

Fuera de alcance

<Lo que explícitamente NO se hace en esta iteración.>

Criterios de finalización

<Ej.: todos los RF con test en verde + demo manual del flujo principal.>

Dudas abiertas

- [NECESITA ACLARACIÓN] <duda>

Paso 3: Clarificación

Ejemplo de Prompt para clarificar una especificación

None

Revisa specs/001-heat-map/spec.md como si fueras un QA muy profesional. Lista: (1) ambigüedades restantes, (2) contradicciones entre requisitos, (3) casos límite no cubiertos, (4) conflictos con docs/constitution.md. No propongas soluciones todavía: solo detecta. Formato: lista numerada.

Paso 4: Planificación (plan.md)

Ejemplo de Prompt para crear planificación de la especificación

None

Lee docs/constitution.md y specs/001-heat-map/spec.md. NO escribas código. Genera specs/001-heat-map/plan.md con: qué archivos se crean o modifican y qué responsabilidad tiene cada uno, qué funciones puras de lógica se necesitan (con "hoy" como parámetro), algoritmo del mapa en pseudocódigo, cómo se pinta en la interfaz, decisiones técnicas justificadas (y su alternativa descartada) y estrategia de tests con node --test. Todo debe respetar la constitución y cubrir todos los RF. Marca qué RF cubre cada parte.

Paso 5: Tareas (tasks.md)

Ejemplo de Prompt para crear las tareas a partir de la planificación

None

A partir de spec.md y plan.md, genera specs/001-heat-map/tasks.md: tareas pequeñas (máx. 20-30 min cada una), en orden de dependencia, cada una con los RF que cubre y una línea "Hecho cuando:" verificable. Usa checkboxes.

Paso 6: Implementación

Ejemplo de Prompt para implementar las tareas

None

Implementa SOLO la tarea T1 de specs/001-heat-map/tasks.md, siguiendo plan.md y la constitución. Escribe primero los tests, luego el código. Ejecuta node --test y muéstrame el resultado. Al terminar: marca T1 en tasks.md, indica qué RF cubre y PÁRATE. No empieces T2.

Paso 7: Validación

Ejemplo de Prompt para validar la especificación implementada

None

Recorre specs/001-heat-map/spec.md requisito por requisito. Para cada uno indica qué test lo cubre y el resultado de ejecutarlo. Los RF de interfaz

que no se puedan testear con `node --test`, verifícalos con Chrome DevTools (incluida la vista móvil). Si algún RF no está cubierto o falla, dilo claramente. Después comprueba los criterios de finalización y dame un veredicto: ¿la spec está cumplida?

Nota: el paso de Clarificación puede realizarse nuevamente luego de los pasos 4 y 5 si algo no está claro.

Automatización de SDD mediante Skills y Comandos

Skill	
/sdd	Flujo y plantilla de la spec

Comando	Ejemplo de Uso	Fase	Agente
/sdd-constitution	/sdd-constitution	Constitución	plan
/sdd-spec	/sdd-spec 002-nombre idea...	Especificación	plan
/sdd-clarify	/sdd-clarify 002-nombre	Clarificación	plan
/sdd-plan	/sdd-plan 002-nombre	Planificación	plan
/sdd-tasks	/sdd-tasks 002-nombre	Tareas	plan
/sdd-implement	/sdd-implement 002-nombre T3	Implementación	build
/sdd-validate	/sdd-validate 002-nombre	Validación	build
/sdd-change	/sdd-change 002-nombre requisito...	Mantenimiento	plan
/sdd-status	/sdd-status 002-nombre	Fase actual y siguiente paso de una spec	plan

.agents/skills/sdd/SKILL.md

```
None
---
name: sdd
description: Úsala siempre que trabajes con Spec-Driven Development en este proyecto
(docs/constitution.md o cualquier archivo dentro de specs/) - redactar, revisar o cambiar
specs, planes y tareas, o implementar y validar tareas de una spec.
---
```

```
# Spec-Driven Development (SDD)

## Flujo
Constitución → Spec → Clarificación → Plan → Tareas → Implementación → Validación → Cambio.

- Nunca pases a la siguiente fase sin la aprobación explícita del usuario.
- La spec manda: si algo no está en la spec, no se implementa. Si falta una decisión, para y pregunta.
- Un cambio de requisitos se hace primero en la spec, luego en el plan y las tareas, y por último en el código.
- Cada spec vive en su carpeta: `specs/NNN-nombre/` con `spec.md`, `plan.md` y `tasks.md`.
- Al terminar cada fase, actualiza `MEMORY.md`.

## Plantilla de spec (spec.md)
```
Spec NNN - <Nombre>

Estado: borrador | aprobada | implementada

Contexto y objetivo
Usuarios
Historias de usuario
- HU-1. Como <rol>, quiero <acción> para <beneficio>.
Definiciones (solo si hay términos que puedan interpretarse de varias formas)
Requisitos funcionales
Requisitos no funcionales
Casos límite
Fuera de alcance
Criterios de finalización
Dudas abiertas
- [NECESITA ACLARACIÓN] <duda>
```

La spec describe el QUÉ y el POR QUÉ. Nada de stack, arquitectura ni nombres de archivos.

## Requisitos en EARS (en español)
- RF-x: CUANDO <evento>, EL SISTEMA <respuesta>.
- RF-x: SI <condición no deseada>, ENTONCES EL SISTEMA <respuesta>.
- RF-x: MIENTRAS <estado>, EL SISTEMA <respuesta>.
- RF-x: EL SISTEMA <comportamiento permanente>.

Cada RF debe ser verificable: nada de "rápido", "intenso" o "bonito" sin un criterio medible.

## Plan (plan.md)
Archivos y responsabilidades · Funciones puras (con "hoy" como parámetro) · Algoritmo en pseudocódigo · Interfaz · Decisiones justificadas con su alternativa descartada · Estrategia de tests con `node --test`. Indica qué RF cubre cada parte.

## Tareas (tasks.md)
```
[] **Tn. <Descripción>.** RF-x, RF-y
- Hecho cuando: <comprobación verificable>.
```

Máximo 20-30 min por tarea, en orden de dependencia. Si salen más de 10, propón dividir la spec.

## Implementación
Una sola tarea cada vez: tests primero (en rojo), después el código, `node --test` en verde, marcar la tarea y parar.
```

.opencode/commands/sdd-constitution.md

None

description: SDD · Propone la constitución del proyecto (principios innegociables)
agent: plan

Vamos a crear (o revisar, si ya existe) docs/constitution.md. Usa la skill sdd.
Antes de proponer nada, lee AGENTS.md, MEMORY.md y el código del proyecto.
Contexto adicional: \$ARGUMENTS

Proponme 6 principios innegociables, cortos y verificables, que cubran:
simplicidad del stack, relación entre spec y código, separación entre lógica
e interfaz, política de tests, protección de los datos del usuario e idioma
del código y los textos. Máximo 15 líneas.

NO escribas el archivo todavía: espera mi aprobación.

.opencode/commands/sdd-spec.md

None

description: SDD · Entrevista y genera la spec (uso - /sdd-spec 002-nombre idea inicial)
agent: plan

NO escribas código en ningún momento. Lee docs/constitution.md y MEMORY.md, y usa la
skill sdd.

Carpeta de la spec: specs/\$1/

Idea inicial (el texto que va después del nombre de la carpeta): \$ARGUMENTS

Tu trabajo:

1. Hazme preguntas de UNA en UNA para eliminar ambigüedades (casos límite, errores, qué queda fuera de esta versión). Máximo 5 preguntas.
2. Con mis respuestas, genera specs/\$1/spec.md siguiendo la plantilla de la skill sdd, con los requisitos en EARS y "Estado: borrador".
3. Solo el QUÉ y el POR QUÉ: nada de stack, arquitectura ni nombres de archivos.

.opencode/commands/sdd-clarify.md

None

description: SDD · Revisa la spec como un QA (solo detecta, no resuelve)
agent: plan

Revisa specs/\$1/spec.md como si fueras un QA muy profesional. Usa la skill sdd.
Lista:

1. Ambigüedades restantes (requisitos que no se pueden verificar).
2. Contradicciones entre requisitos.
3. Casos límite no cubiertos.
4. Conflictos con docs/constitution.md.

No propongas soluciones todavía: solo detecta. Formato: lista numerada.

.opencode/commands/sdd-plan.md

```
None
---
description: SDD · Genera el plan técnico de una spec aprobada
agent: plan
---
Lee docs/constitution.md, AGENTS.md y specs/$1/spec.md. Usa la skill sdd.
NO escribas código.

Genera specs/$1/plan.md con: archivos que se crean o modifican y la
responsabilidad de cada uno, funciones puras de lógica (con "hoy" como
parámetro), algoritmo en pseudocódigo, cómo se pinta en la interfaz,
decisiones técnicas justificadas (con su alternativa descartada) y estrategia
de tests con node --test.

Todo debe respetar la constitución y cubrir todos los RF. Marca qué RF cubre
cada parte. Si la spec no está aprobada o tiene dudas abiertas, para y avísame.
```

.opencode/commands/sdd-tasks.md

```
None
---
description: SDD · Divide el plan en tareas pequeñas y verificables
agent: plan
---
A partir de specs/$1/spec.md y specs/$1/plan.md, genera specs/$1/tasks.md
siguiendo el formato de la skill sdd:
- Tareas pequeñas (máx. 20-30 min cada una), en orden de dependencia.
- Cada una con los RF que cubre y una línea "Hecho cuando:" verificable.
- Checkboxes.
- Intenta que no sean más de 10: si salen más, propón dividir la spec.
```

.opencode/commands/sdd-implement.md

```
None
---
description: SDD · Implementa UNA tarea, tests primero (uso - /sdd-implement 002-nombre
T3)
agent: build
---
Implementa SOLO la tarea $2 de specs/$1/tasks.md, siguiendo specs/$1/plan.md,
docs/constitution.md y la skill sdd.

1. Escribe primero los tests y comprueba que fallan.
2. Escribe el código hasta que pasen.
3. Ejecuta node --test y muéstrame el resultado.
4. Marca $2 como hecha en tasks.md e indica qué RF cubre.

Después PÁRATE. No empieces la siguiente tarea.
```

.opencode/commands/sdd-validate.md

```
None
---
description: SDD · Valida la spec RF por RF (tests + Chrome DevTools)
agent: build
---
Recorre specs/$1/spec.md requisito por requisito. Para cada RF indica qué test
lo cubre y el resultado de ejecutarlo con node --test.

Los RF de interfaz que no se puedan testear con node --test, verifícalos con
el MCP de Chrome DevTools (incluida la vista móvil de 375 px).

Si algún RF no está cubierto o falla, dilo claramente. NO arregles nada todavía.
Después comprueba los criterios de finalización y dame un veredicto:
¿la spec está cumplida?
```

.opencode/commands/sdd-change.md

```
None
---
description: SDD · Nuevo requisito - primero la spec, luego el código
agent: plan
---
Nuevo requisito para la spec specs/$1/ (el texto que va después del nombre de
la carpeta): $ARGUMENTS

NO toques código. Usa la skill sdd.
1. Actualiza specs/$1/spec.md: nuevo RF (o cambio de uno existente) en EARS,
sus casos límite y lo que queda fuera de alcance.
2. Indica qué partes de plan.md y tasks.md habría que cambiar después.
3. Muéstrame el diff de la spec y espera mi aprobación.
```

.opencode/commands/sdd-status.md

```
None
---
description: SDD · Dónde estamos - fase actual y siguiente paso de una spec
agent: plan
---
Lee specs/$1/ (spec.md, plan.md y tasks.md, los que existan) y MEMORY.md.

Dime en pocas líneas:
1. En qué fase del flujo SDD está esta spec y su estado.
2. Tareas hechas y pendientes (x de y).
3. El siguiente paso exacto, con el comando /sdd-* que debo ejecutar.

No modifiques ningún archivo.
```

Súmate a nuestro directo del día 3

[IR AL DIRECTO](#)